



Department of Computer Science & Engineering
CSPIT, CHARUSAT

A deep dive into immutability, memory, and mutable string alternatives in
Java

- 1 The String Class & Immutability
- 2 String Creation & Memory
- 3 Why Immutability?
- 4 Essential String Methods
- 5 String Comparison
- 6 Why We Need a Mutable Alternative
- 7 StringBuffer
- 8 StringBuilder
- 9 Comparisons, Performance & Best Practices
- 10 Practice Exercises
- 11 Wrapping Up

Definition

A **String** is a sequence of characters treated as a single object.

- In Java, String is a class (`java.lang.String`), not a primitive type.
- Every string literal or string expression is a String object.
- **The most important property: it is IMMUTABLE** — once created, its content can never change.

`java.lang.String`: Declared `final` — cannot be subclassed

Backing store: `char[]` / `byte[]` array

Interfaces: `Comparable<String>`, `CharSequence`

Creation: literal & `new` keyword

Literal Syntax

```
String s1 = "Hello";
```

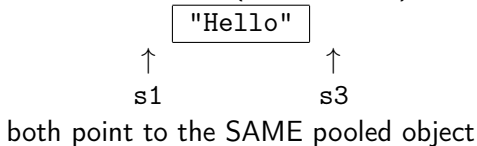
- Checks the String Pool first
- Reuses existing object if value matches
- Memory efficient

new Keyword

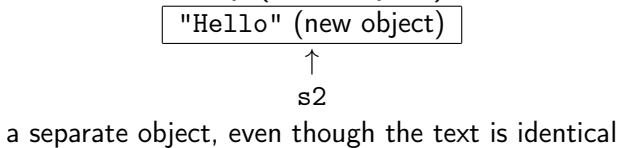
```
String s2 = new String("Hello");
```

- Always creates a new object on the heap
- Ignores the pool, even for identical values
- Slightly less memory efficient

String Pool (inside Heap)



Heap (outside pool)



Security

Strings often carry sensitive data (file paths, network connections, credentials). Immutability prevents tampering after creation.

String Pool Works

Pooling/sharing literals is only safe if the shared object can never change underneath another reference.

HashCode Caching

String's `hashCode` is computed once and cached — safe only because the content never changes. Speeds up `HashMap/HashSet` lookups.

Thread Safety

An immutable object can be freely shared across threads with no synchronization needed.

```
String s = "Hello";  
s.concat(" World");  
System.out.println(s);           // Hello (unchanged!)  
  
s = s.concat(" World");  
System.out.println(s);           // Hello World (NEW object!)
```

- `concat()` does **not** modify the original object — it returns a brand-new `String`.
- Unless the result is reassigned, the original reference is completely unaffected.
- **Common beginner bug:** calling a `String` method and forgetting to store its return value.

`length()` Number of characters
`charAt(i)` Character at index `i`
`indexOf(x)` First index of `x`, or `-1`
`substring(a,b)` Slice from `a` to `b-1`
`concat(str)` Appends `str`, returns new `String`
`replace(a,b)` Replaces all `a` with `b`

`toUpperCase()` Converts to upper case
`toLowerCase()` Converts to lower case
`trim()` / `strip()` Removes leading/trailing spaces
`split(regex)` Breaks into a `String[]`
`equals(obj)` Content-based comparison
`compareTo(str)` Lexicographic comparison

```
String str = "CHARUSAT";

System.out.println(str.length());    // 8
System.out.println(str.charAt(2));    // A
System.out.println(str.indexOf('U')); // 3
System.out.println(str.indexOf('Z')); // -1 (not found)
```

Index positions are zero-based: C(0) H(1) A(2) R(3) U(4) S(5) A(6)
T(7)

```
String s = "Java Programming";

System.out.println(s.substring(5));    // Programming
System.out.println(s.substring(0, 4)); // Java
System.out.println(s.concat(" Language")); // Java Programming
                                         Language
System.out.println(s.replace('a', '@')); // J@v@ Progr@mming
```

- `substring(a)` returns everything from index `a` to the end.
- `substring(a, b)` returns index `a` up to (but not including) index `b`.
- Every method here returns a NEW String — `s` itself is never modified.

```
String s = " Hello World ";  
  
System.out.println(s.trim());           // "Hello World"  
System.out.println(s.toUpperCase());     // " HELLO WORLD "  
  
String[] words = s.trim().split(" ");  
// words = { "Hello", "World" }
```

== — Reference Comparison

- Checks whether two variables point to the **exact same object** in memory
- True only if both references are identical
- Does NOT compare the actual characters

.equals() — Content Comparison

- Overridden in the String class to compare actual character sequences
- True whenever the text content matches, regardless of memory location
- This is what you almost always want to use

```
String a = "test";  
String b = "test";  
String c = new String("test");  
  
System.out.println(a == b);      // true  -- both from the pool  
System.out.println(a == c);      // false -- c is a separate heap  
    object  
System.out.println(a.equals(c)); // true  -- same characters
```

Rule of thumb: never use `==` to compare String content — always use `.equals()`

Returns an int describing lexicographic (dictionary) order.

0 → the strings are equal

Negative → calling string comes before the argument

Positive → calling string comes after the argument

Comparison is based on Unicode value differences of the first mismatched characters.

```
String s1 = "Apple";  
String s2 = "Banana";  
  
System.out.println(s1.compareTo(s2));  
// negative -- 'A' (65) comes before 'B' (66)
```

```
String result = "";  
for (int i = 0; i < 1000; i++) {  
    result = result + i; // a NEW String object is created every  
        iteration!  
}
```

Because String is immutable, every "+" concatenation discards the old object and allocates a brand new one — 1000 iterations can mean roughly 1000 throwaway objects, wasting time and memory.

StringBuffer

A `StringBuffer` represents a mutable sequence of characters — it can grow, shrink, and change in place.

- Lives in `java.lang`, just like `String`.
- Every modifying method is synchronized, making it **thread-safe**.
- Best suited when multiple threads may modify the same character sequence.

`append(x)` Adds x to the end
`insert(i, x)` Inserts x at index i
`delete(a,b)` Removes characters
a..b-1
`deleteCharAt(i)` Removes
character at i

`reverse()` Reverses the sequence
`replace(a,b,s)` Replaces a..b-1
with s
`capacity()` Current buffer
capacity
`charAt(i)` / `setCharAt(i,c)`
Read / write a single
character

```
StringBuffer sb = new StringBuffer("Hello");  
  
sb.append(" World"); // sb = "Hello World"  
sb.insert(5, ","); // sb = "Hello, World"  
sb.reverse(); // sb = "dlroW ,olleH"  
  
System.out.println(sb); // dlroW ,olleH
```

Notice: sb itself was modified in place — no new object was created at each step.

StringBuilder

`StringBuilder` offers the exact same API as `StringBuffer`: `append`, `insert`, `delete`, `reverse`, `replace`, and more.

- Introduced in Java 5 (JDK 1.5) as a faster, single-threaded alternative.
- Its methods are **NOT** synchronized — no thread-safety overhead.
- The recommended default choice for string building in ordinary, single-threaded code.

```
StringBuilder sb = new StringBuilder();  
  
for (int i = 0; i < 5; i++) {  
    sb.append(i).append(","); // method chaining  
}  
  
System.out.println(sb.toString()); // 0,1,2,3,4,
```

- `append()` returns the `StringBuilder` itself, enabling fluent method chaining.
- Only ONE object is created and reused across the entire loop.

Feature	StringBuffer	StringBuilder
Thread-Safe	Yes (synchronized)	No
Performance	Slower	Faster
Introduced	JDK 1.0	JDK 1.5
Use Case	Multi-threaded	Single-threaded

Feature	String	StringBuffer	StringBu
Immutable	Yes	No	No
Thread-Safe	Yes	Yes	No
Performance	Slow (for changes)	Moderate	Fast
Memory Efficient	Good (pool)	Moderate	Moderate
Use Case	Fixed text	Multi-threaded edits	Single-threaded

Concatenating 50,000 Strings

- Numbers are illustrative, not measured — the point is the shape of the gap.
- String concatenation in a loop scales poorly as size grows.
- `StringBuilder` edges out `StringBuffer` thanks to skipping synchronization.

Method	Relative Time
<code>String concat (+)</code>	Very High
<code>StringBuffer</code>	Medium-High
<code>StringBuilder</code>	Low

- ❶ **String** — Text is fixed, or changed rarely (config values, keys, constants).
- ❷ **StringBuilder** — Frequent edits in ordinary, single-threaded code. The default choice for building text.
- ❸ **StringBuffer** — Frequent edits shared across multiple threads that may run concurrently.

- **Forgetting immutability:** calling `str.replace(...)` or `str.trim()` and expecting `str` itself to change — the return value must be captured.
- Using `==` to compare `String` content instead of `.equals()` — works "by accident" for pooled literals but fails for new `String(...)`.
- Assuming `StringBuilder` is thread-safe — it is not; use `StringBuffer` (or external synchronization) when threads share it.
- Building large strings with `+` inside a loop instead of `StringBuilder` — a classic performance mistake.
- Confusing `substring(a, b)`'s end index: `b` is EXCLUSIVE, not inclusive.

```
String s1 = "Java";  
String s2 = "Java";  
String s3 = new String("Java");  
  
System.out.println(s1 == s2);  
System.out.println(s1 == s3);  
System.out.println(s1.equals(s3));
```

Think about the String Pool vs the heap before checking the answer.

Answer: true, false, true

Write a Java program that reverses a given String using StringBuilder — **WITHOUT** calling its built-in `reverse()` method.

- Hint: iterate through the original string from the last character to the first.
- Use `sb.append(ch)` inside the loop to build the reversed sequence.
- Bonus: compare the character count of your loop-based version against `sb.reverse()` for correctness.

- `String` is immutable — every modifying method returns a new object.
- `String` literals are shared via the String Pool; `new String()` always creates a fresh heap object.
- Use `.equals()` for content comparison, never `==` (except when you deliberately want reference comparison).
- `StringBuffer` is mutable and thread-safe (synchronized); `StringBuilder` is mutable and faster (not synchronized).
- Prefer `StringBuilder` for everyday string building; reach for `StringBuffer` only in multi-threaded contexts.

Thank You

Questions & Discussion

Department of Computer Science & Engineering
CSPIT, CHARUSAT